

Cost Controls

Two mechanisms: a **pre-flight estimate with a hard gate** (no surprise spend) and a **per-member dashboard** (manage spend across the agency). Cost control here means dials and a draft-then-finalise flow — never a cap on output quality.

**Real numbers (June 2026
ballpark — store in `rate_card`,
confirm in each provider console)**

Item	Rate	Notes
Claude Sonnet 4.6 (producer default)	\$3 / \$15 per 1M tok	Sweet spot for RAG + content
Claude Opus 4.8 (premium toggle)	\$5 / \$25 per 1M tok	effort=high for flagship/complex briefs
Claude Haiku 4.5 (ingestion/tagging)	\$1 / \$5 per 1M tok	High-volume, low-complexity

Item	Rate	Notes
Prompt caching / Batch	–90% cached input · –50% batch	Cache the system prompt + client profile
Embeddings	~cents per 1M tok	Dedicated embeddings model (e.g. Voyage); match vector() dims
ElevenLabs TTS	~1 credit/char (Flash 0.5)	~700 chars for a 45s VO = cents marginal
ElevenLabs Music / SFX	credit- based	~45s track = cents; SFX off by default
HeyGen Avatar V API	\$0.05/sec (\$3/min)	Pay-as-you-go; biggest single line if avatar runs long
Higgsfield DoP b- roll	credit/clip, tier-	lite << turbo << preview – the

Item	Rate	Notes
	dependent	biggest dial
Magnific Skin Enhancer	~\$0.08 (2K) / \$0.16 (4K) per image	Selective, hero frames only
Shotstack render	~\$0.20–0.40 per min @1080p	4K more; low-res draft cheaper

Worked example — one 45s video (5 scenes, avatar ~25s, music on, SFX off, 2 hero frames, 1080p, Sonnet producer w/ caching)

```

Producer (Sonnet 4.6, cached)
~$0.10
Embeddings / retrieval
<$0.01
TTS (~700 chars)
~$0.02
Music (~45s)
~$0.10
HeyGen a-roll (25s × $0.05)
~$1.25

```

```
Higgsfield b-roll (5 × ~$0.30)
~$1.50
Magnific (2 frames × ~$0.10)
~$0.20
Shotstack (draft ~$0.10 + 1080p)
~$0.35
```

```
TOTAL ≈
$3.50 per finished video
```

4K + preview-tier b-roll pushes this to ~\$6–8; lite tier + shorter avatar + 1080p pulls it to ~\$2. **The render dials move the bill; the LLM doesn't.**

Where the money is — and the Opus question, answered

The producer LLM is **~3% of a ~\$3.50 video**. Switching the producer from Sonnet 4.6 to Opus 4.8 adds roughly **\$0.10 per video** — a rounding error next to HeyGen and Higgsfield. So the policy is: **default Sonnet 4.6** (excellent for this, 40% cheaper than Opus), offer **Opus 4.8 (effort=high) as an optional premium pass** for flagship clients or tricky briefs, use **Haiku 4.5** for ingestion/tagging, and **always**

prompt-cache the system prompt + client profile (they repeat every turn → -90% on the dominant input cost). Then spend the real cost-control effort on the dials below, not the model.

Why a hard gate is possible here

Unlike cloud billing (e.g. Google Cloud budgets are alert-only — no hard cap, because the provider owns the spend trigger), **we own the trigger**. A production cannot call a paid API until it passes the budget check. So an over-budget run is *blocked*, not just emailed about after the money's gone.

Estimation algorithm

The cheap `script.generate` stage produces the cost-driving metrics. Map them to `rate_card` rows (prices live in the table, never in code):

```
estimate_cents =  
    tts:      char_count      ×  
    rate(elevenlabs, tts, char)      [0  
    if uploaded audio]
```

```

+ music:    duration_s          ×
rate(elevenlabs, music_v2, second) [0
if music off]
+ ambient: duration_s          ×
rate(elevenlabs, sfx, second)      [0
if ambient off]
+ aroll:    duration_s/60       ×
rate(heygen, avatar_v, minute)
+ broll:    scene_count         ×
rate(higgsfield, <tier>, clip)
+ enhance: hero_frame_count     ×
rate(magnific, skin_enhancer, image)
[selective]
+ draft:    duration_s/60       ×
rate(shotstack, render_minute, low)
+ final:    duration_s/60       ×
rate(shotstack, render_minute,
<resolution>)
+ script:   tokens              ×
rate(anthropic, opus, token)
[pennies]

```

Store on `productions.estimate_cents`. Show the operator the **breakdown by stage** at GATE 1.

Dials (cost-sensitive AND world-class)

Same brief, lower cost — surfaced as toggles the operator can pull before approving:

Dial	Effect
Higgsfield tier (preview → turbo → lite)	Biggest b-roll lever
Magnific hero-frame count (fewer enhanced frames)	Trims realism-pass spend; keep at least the influencer's face frames
Resolution (4K → 1080p)	Lowers final render cost
Pop-ups / music / ambient off	Removes those stage costs
Fewer scenes	Fewer b-roll clips
Uploaded audio	Zero TTS spend
Shorter cut	Scales most stages down

The estimator can show "this config = R/\$X; here's the path to your target." You're showing the trade, not lowering the ceiling.

Budget gate (GATE 1 logic)

```
remaining = budget.limit_cents -  
budget.spent_cents    -- for the run's  
user + client scope  
if estimate_cents > remaining:  
    if budget.hard_gate: block;  
    require larger budget OR a cheaper dial  
    config  
    else: warn; allow  
with explicit override (logged)  
else: proceed
```

Evaluate the tightest applicable scope (user, then client, then team). On completion, increment `budgets.spent_cents` by `actual_cents`.

Dashboard (per member, per client)

Source of truth: `production_steps` joined to `productions`, `users`, `clients`.

Show:

- **Spend by member / client / period**, with **estimate-vs-actual variance** (catches drift).

- **Budget status** per member and per client: limit, spent, remaining, gate on/off.
- **Cost per production**, drillable to per-stage breakdown.
- **Most expensive stages** across the agency (where to tune the rate card or default dials).

Admins set budgets and edit the rate card; producers see their own spend and run estimates.

All figures render in the user's

`display_currency` (ZAR/USD/both) per the currency section

below — base stays USD.

Currency — USD base, local display (ZAR and beyond)

Every provider bills in **USD**, so USD is the **base currency and single source of truth**:

the `rate_card`, `production_steps.cost_cents`, and every stored cost stay in USD cents.

Local currency is a **display layer** — never store a converted amount as the canonical value, or FX drift and rounding will corrupt your spend history.

Display. Each user (and each tenant's default) has a `display_currency` and a "show both" option. ZAR-first agencies see **R64.20 (\$3.50)**; toggle to ZAR-only or USD-only.

Two rates, used deliberately:

- **Live rate** for *forward-looking* views — estimates, budgets, the live cost meter. Pull from a reputable FX source (e.g. Frankfurter/ECB, exchangerate-api, openexchangerates), cache it, and serve from Vercel **Edge Config** for sub-ms reads. A daily refresh on an Inngest cron is plenty for cost display — don't fetch per page load.
- **Snapshotted rate** for *historical* records — when a production completes, store the `fx_rate_snapshot` and currency used, so its ZAR cost doesn't silently change next week when the rand moves. Past spend must be stable and auditable.

Budgets can be set in the agency's currency (ZAR), but the **gate evaluates in USD base** (convert the budget to USD at the current rate). Flag this clearly in the UI: because ZAR is

volatile, a rand budget's USD headroom shifts with the exchange rate — always show the rate and its timestamp so it's transparent, never a surprise.

Honest note: the local figure is an **indicative conversion** for decisions and budgeting; providers still charge USD to whatever card is connected. It's what your team sees and plans in, not necessarily the exact rand your bank debits.

Scaling international: the model is currency-agnostic — add a display currency and its USD pair and any market works. A per-tenant default currency means a UK or AU agency onboarded later sees GBP/AUD natively while USD stays the base everywhere.

```
-- cached rates for reads (mirror the
current set into Edge Config for speed)
create table fx_rates (
  base      text not null default
'USD',
  quote     text not null,
-- 'ZAR','GBP','AUD',...
  rate      numeric not null,
-- 1 USD = rate * quote
  fetched_at timestamptz not null
```

```
default now(),
  primary key (base, quote, fetched_at)
);
alter table users      add column
display_currency text default 'USD';
alter table users      add column
show_both           boolean default true;
alter table clients    add column
default_currency text default 'USD';
alter table productions add column
fx_rate_snapshot numeric;  -- locked
at completion
alter table productions add column
display_currency text;
alter table budgets     add column
currency               text default 'USD'; -
- limit's currency; gate in USD
```

Connected accounts — whose spend is it?

Provider credentials are per tenant (the Connect Tools step). For **Iteration 1**, GAS connects GAS's ElevenLabs/Higgsfield/HeyGen accounts, so GAS pays the providers and the dashboard tracks GAS team + per-client spend exactly as below. For **market customers** (Iteration 2), each connects their own accounts —

they pay the providers directly, while GAS layers a platform/markup fee. Either way the estimate and dashboard work unchanged (usage is metered regardless of whose card the provider bills); only the payer differs.

Keeping the rate card honest

Provider prices change. The `rate_card` is operator-editable (admin UI), versioned via `effective_from`. Reconcile periodically: compare logged `actual_cents` (from real provider usage in responses) against estimates; if variance creeps up, the prices need refreshing. Never hardcode a price in application code.